

SARDAR VALLABHBHAI NATIONAL INSTITUTE OF TECHNOLOGY, SURAT
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CS103: DESIGN AND ANALYSIS OF ALGORITHMS
M.Tech. (DS) Ist Year
Lab Assignment

Assignment 1:

Find the closest pair of points in a two-dimensional plane using the Divide and Conquer technique.

Tasks:

- Implement the brute-force approach.
- Implement the divide-and-conquer approach.
- Generate sets of points of different sizes.
- Measure execution times of both approaches.
- Compare their empirical performance.
- Analyze the theoretical complexities.

Assignment 2:

This assignment concerns creating a data set consisting of a vector of integer values. Create this data set - consisting of at least a million integer values in a vector using the functions and appropriate additional functions as follows: Use the random number generator functions `srand48()`, `lrand48()` and/or `nrnd48()` to populate a vector with a million entries to serve as an input data set to test your program. The vector must consist of only the positive integers. Now, write another function to print the number of duplicates found in the input, if any and yet another function to round off the duplicate to the nearest free integral value. Edit your program to measure the time required to remove the duplicates. Discuss various approaches to time the programs, including the utilities available in Linux. Use an approach that gives you the finest resolution in time.

Assignment 3:

Now, assume that the new requirement is that (if not already taken care of) - your program must output the time in microseconds. How do we achieve the same? What approach are you able to devise as the solution to this question?

Assignment 4:

Given an unsorted array `arr[]` and an index `i`, find the element that would be at index `i` if the array were sorted. Can you use a priority queue to solve this problem? If yes, write the algorithm, implement the solution, and analyze its time complexity. Note that the priority queue-based implementation tends to have higher complexity when the index of `x` is closer to `n` (the size of the array). Alternatively, can you use the partition routine from the Quicksort algorithm to solve the same problem more efficiently? If yes, write the algorithm, implement it, and analyze its time complexity. Now compare the three algorithms based on time complexity.

Assignment 5:

Find the cost of execution of the following code snippet by writing AN appropriate program, using the timing program and compare with the asymptotic values you obtained in the class:

(a) Code1

```
for (i = 1; i<=n; i++)  
    for (j = 1; j<=i; j++)  
        x = x + 1;
```

(b) Code2

```
j=n  
while (j >= 1){  
    for (i = 1; i<=j; i++)  
        x = x + 1;  
    j = n/2;  
}
```

(c) Code3

```
for (i = 1; i<=n; i++)  
    for (j = 1; j<=i; j++)  
        for (k=1; k<=i; k++)  
            x = x + 1;
```

(d) Code4

```
i=n  
while (i >= 1){  
    for (j = 1; j<= n; j++)  
        x = x + 1;  
    i = i /2;  
}
```

Assignment 6:

Implement large integer multiplication using the Karatsuba Divide and Conquer algorithm.

Task:

- Implement conventional multiplication.
- Implement Karatsuba multiplication.
- Test with integers having increasing numbers of digits.
- Measure execution time.
- Compare the empirical performance of both methods.
- Analyze the theoretical complexity.

Assignment 7:

Fast Exponentiation:

Aim: Compute an efficiently using the Divide and Conquer technique.

Tasks:

- Implement conventional exponentiation.
- Implement recursive fast exponentiation.
- Test for increasingly large values of n.
- Measure execution time.
- Compare the two approaches.
- Verify **$O(\log n)$** complexity of fast exponentiation.

Assignment 8:

Counting Inversions:

Aim: Design an efficient Divide and Conquer algorithm to count the number of inversions in an array.

Tasks:

- Implement the inversion-counting algorithm using Merge Sort.
- Implement the naïve $O(n^2)$ approach for comparison.
- Measure execution time for different input sizes.
- Compare the two algorithms using a performance graph.
- Analyze the theoretical and empirical complexities.